

Data Collection – Selenium

Introduction to Selenium

Selenium Python bindings provide a **simple and intuitive API** to write automated browser tests and perform web automation tasks.

Using Selenium WebDriver, you can:

- Control real web browsers
- Simulate user actions (click, scroll, type, drag)
- Interact with JavaScript-heavy and dynamic websites
- Build functional/acceptance tests
- Automate tasks that cannot be handled by static HTML parsers like BeautifulSoup

Selenium Python API gives full access to all Selenium WebDrivers such as:

- **Chrome**
- **Firefox**
- **Edge**
- **Internet Explorer** (legacy)
- **Remote WebDriver** for distributed testing

Python 3.5+ is fully supported.

Installing Selenium (Python Bindings)

Selenium Python bindings can be installed easily using pip.

Install using pip

pip install selenium

What You Need After Installation

After installing Selenium, you must download the appropriate browser driver:

- **Chrome → ChromeDriver**
- **Firefox → GeckoDriver**

- **Edge → Edge WebDriver**

These drivers act as a **bridge** between Selenium and the browser.

Place the driver in:

- Your project folder
- OR add it to your system PATH

Basic Browser Launch Example

A simple example to open a webpage using Selenium:

```
from selenium import webdriver  
driver = webdriver.Chrome()  
driver.get("https://example.com")
```

This:

- Launches Chrome using ChromeDriver
- Opens the given URL
- Creates a browser session you can control programmatically

Simple Usage of Selenium (Using Chrome WebDriver)

Once Selenium Python bindings are installed, you can start using Selenium to automate browser tasks.

Below is a simple script that:

- Opens Chrome
- Navigates to python.org
- Searches for “pycon”
- Verifies results
- Closes the browser

```
from selenium import webdriver

from selenium.webdriver.common.keys import Keys

from selenium.webdriver.common.by import By

# Create Chrome WebDriver instance

driver = webdriver.Chrome()

# Open a webpage

driver.get("http://www.python.org")

# Check if page title contains "Python"

assert "Python" in driver.title

# Locate the search box by the name attribute

elem = driver.find_element(By.NAME, "q")

# Clear the field, type a query, and press Enter

elem.clear()

elem.send_keys("pycon")

elem.send_keys(Keys.RETURN)

# Verify that results are displayed

assert "No results found." not in driver.page_source

# Close the browser window

driver.close()
```

Example Explained

Imports

```
from selenium import webdriver
```

```
from selenium.webdriver.common.keys import Keys
```

```
from selenium.webdriver.common.by import By
```

- `webdriver` → Provides browser drivers (Chrome, Firefox, Edge, etc.).
- `Keys` → Lets us send special keyboard keys (ENTER, TAB, ESC, etc.).
- `By` → Used to locate HTML elements.

Creating the Chrome WebDriver Instance

```
driver = webdriver.Chrome()
```

- This launches **Google Chrome** using **ChromeDriver**.
- Ensure ChromeDriver is installed and available in PATH or the script folder.

Opening a Webpage

```
driver.get("http://www.python.org")
```

- WebDriver waits until the page's **onload event** completes.
- Note: If the site loads a lot of AJAX data, some elements may still load after onload.

Checking the Page Title

```
assert "Python" in driver.title
```

This verifies that the page loaded correctly.

Finding an Element

```
elem = driver.find_element(By.NAME, "q")
```

- `find_element` locates the **first matching element**.
- `By.NAME` means: “look for an HTML element with a name attribute of 'q'”.

Typing into a Field

```
elem.clear()
```

```
elem.send_keys("pycon")
```

```
elem.send_keys(Keys.RETURN)
```

- `clear()` removes existing text
- `send_keys()` types into the field
- `Keys.RETURN` simulates pressing ENTER

Verifying Search Results

```
assert "No results found." not in driver.page_source
```

This checks that Python.org returned some results.

Closing the Browser

```
driver.close()
```

- `close()` → closes the current tab
- `quit()` → closes the entire browser session

If only one tab is open, both behave similarly.

Navigating and Interacting with Web Pages in Selenium

Navigation

The first step in Selenium automation is navigating to a webpage. This is done using the `get()` method:

```
driver.get("http://www.google.com")
```

Selenium waits until the page's **onload** event fires before giving control back to the script.

However, if the site loads data using AJAX, Selenium might not detect full loading. In such cases, explicit waits should be used.

Example

Automating login to a website like Facebook or Gmail:

```
driver.get("https://www.facebook.com")
```

The page loads, but the login form itself may appear a second later due to AJAX. This is where explicit waits are needed.

Interacting with the Page

Selenium interacts with web pages by locating HTML elements and performing actions on them.

Locating Elements

Suppose the page contains:

```
<input type="text" name="passwd" id="passwd-id" />
```

You can locate it using several strategies:

```
element = driver.find_element(By.ID, "passwd-id")
```

```
element = driver.find_element(By.NAME, "passwd")
```

```
element = driver.find_element(By.XPATH, "//*[@id='passwd-id']")
```

```
element = driver.find_element(By.CSS_SELECTOR, "input#passwd-id")
```

Important notes:

- Link text searches must be an exact match.
- If multiple elements match, Selenium returns the first.
- If no element is found, Selenium raises NoSuchElementException.
- Selenium uses an object-based API; all elements share a similar interface.

Example

Locating the search bar on Amazon:

```
search = driver.find_element(By.ID, "twotabsearchtextbox")
```

Typing into Elements

To enter text into a text field:

```
element.send_keys("some text")
```

To use keyboard shortcuts or special keys:

```
element.send_keys(" and some", Keys.ARROW_DOWN)
```

To clear an input field:

```
element.clear()
```

Selenium does not clear text automatically; new content is appended.

Example

Typing a product name into an e-commerce site:

```
search = driver.find_element(By.ID, "twotabsearchtextbox")
```

```
search.clear()
```

```
search.send_keys("laptop")
```

```
search.send_keys(Keys.RETURN)
```

Filling Forms

Interacting With Select Dropdowns

Basic iteration over options:

```
element = driver.find_element(By.XPATH, "//*[@name='name']")
```

```
all_options = element.find_elements(By.TAG_NAME, "option")
```

```
for option in all_options:
```

```
    print("Value is:", option.get_attribute("value"))
```

```
    option.click()
```

Example

Selecting a country from a dropdown in a signup page:

```
country = driver.find_element(By.ID, "country")  
usa = country.find_element(By.XPATH, "//*[@value='US']")  
usa.click()
```

Example HTML:

```
<select name="course" id="course">  
  <option value="bca">BCA</option>  
  <option value="mca">MCA</option>  
  <option value="mba">MBA</option>  
</select>
```

Click Only Specific Value (Example: BCA)

```
element = driver.find_element(By.XPATH, "//*[@name='course']")  
bca_option = element.find_element(By.XPATH, "//*[@value='bca']")  
bca_option.click()
```

Using CSS Selector:

```
driver.find_element(By.CSS_SELECTOR, "option[value='bca']").click()
```

Using the Select Class (Recommended)

```
from selenium.webdriver.support.ui import Select  
  
select = Select(driver.find_element(By.NAME, 'name'))  
select.select_by_index(index)  
select.select_by_visible_text("text")  
select.select_by_value(value)
```

Example

Choosing quantity on an Amazon product page:

```
qty = Select(driver.find_element(By.ID, "quantity"))  
qty.select_by_value("3")
```

Deselecting Options

Works only for multi-select dropdowns:

```
select = Select(driver.find_element(By.ID, 'id'))  
select.deselect_all()
```

Example

Deselecting all selected categories in a multi-select filter:

```
Select(driver.find_element(By.ID, "categories")).deselect_all()
```

Retrieving Options

Default selected options:

```
all_selected_options = select.all_selected_options
```

All available options:

```
options = select.options
```

Example

Reading all available delivery methods:

```
methods = Select(driver.find_element(By.ID, "delivery")).options
```

Submitting the Form

Using a button:

```
driver.find_element(By.ID, "submit").click()
```

Using the submit method:

```
element.submit()
```

submit() only works if the element is inside a <form>.

Example

Submitting a search form on a job website:

```
driver.find_element(By.ID, "search-button").click()
```

Drag and Drop

```
element = driver.find_element(By.NAME, "source")
```

```
target = driver.find_element(By.NAME, "target")
```

```
from selenium.webdriver import ActionChains
```

```
action = ActionChains(driver)
```

```
action.drag_and_drop(element, target).perform()
```

Example

Dragging items on a dashboard (like Trello):

```
action.drag_and_drop(task_card, completed_column).perform()
```

Working with Windows and Frames

Switching Between Windows

```
driver.switch_to.window("windowName")
```

Iterating through all windows:

```
for handle in driver.window_handles:
```

```
driver.switch_to.window(handle)
```

Example

Clicking a link that opens in a new tab (bank statements, PDF downloads).

Switching Frames / iframes

```
driver.switch_to.frame("frameName")
```

Nested frames:

```
driver.switch_to.frame("frameName.0.child")
```

Return to main page:

```
driver.switch_to.default_content()
```

Example

Interacting with a chatbox that loads inside an iframe.

Handling Popup Dialogs (Alerts)

```
alert = driver.switch_to.alert
```

```
alert.accept()
```

```
alert.dismiss()
```

```
alert.text
```

```
alert.send_keys("text for prompt")
```

Example

Confirming a delete action:

Are you sure you want to delete this item?

```
alert.accept()
```

Navigation: History and Location

Navigate to a page:

```
driver.get("http://www.example.com")
```

Navigate back and forward:

```
driver.back()
```

```
driver.forward()
```

Example

Returning to the previous product page during online shopping.

Working with Cookies

Navigate to the domain:

```
driver.get("http://www.example.com")
```

Add a cookie:

```
cookie = {"name": "foo", "value": "bar"}
```

```
driver.add_cookie(cookie)
```

Retrieve cookies:

```
driver.get_cookies()
```

Example

Simulating a logged-in session by adding authentication cookies.

Locating Elements in Selenium

Selenium provides multiple strategies to locate elements on a web page. Choosing the correct locator type is important for building fast, reliable, and stable automation scripts.

To locate a **single element**, Selenium uses:

`find_element`

To locate **multiple elements**, Selenium uses:

`find_elements`

Example:

```
from selenium.webdriver.common.by import By
```

```
driver.find_element(By.XPATH, '//button[text()="Some text"]')
```

```
driver.find_elements(By.XPATH, '//button')
```

The By class defines all the supported locator types:

- ID
- NAME
- XPATH
- LINK_TEXT
- PARTIAL_LINK_TEXT
- TAG_NAME
- CLASS_NAME
- CSS_SELECTOR

Each locator strategy is appropriate for a certain situation.

Locating Elements Using Different Strategies

Locating by ID

Use this when the element has a unique id attribute.
IDs are usually the **fastest and most reliable** locator.

Example HTML:

```
<form id="loginForm">  
  <input name="username" type="text" />  
  <input name="password" type="password" />  
  <input name="continue" type="submit" value="Login" />  
</form>
```

Selenium:

```
login_form = driver.find_element(By.ID, "loginForm")
```

Locating by Name

Use when the element has a name attribute.
If multiple elements share the same name, Selenium returns the **first matching one**.

Example HTML:

```
<input name="username" type="text" />  
<input name="password" type="password" />  
<input name="continue" type="submit" value="Login" />  
<input name="continue" type="button" value="Clear" />
```

Selenium:

```
username = driver.find_element(By.NAME, "username")  
password = driver.find_element(By.NAME, "password")  
continue_button = driver.find_element(By.NAME, "continue")
```

Locating by XPath

XPath is a powerful locator when:

- ID or Name is missing
- You need to locate elements using complex conditions
- You want to locate elements based on relationships (parent/child)

Example HTML:

```
<form id="loginForm">  
  <input name="username" type="text" />  
  <input name="password" type="password" />  
  <input name="continue" type="submit" value="Login" />  
  <input name="continue" type="button" value="Clear" />  
</form>
```

Examples:

```
driver.find_element(By.XPATH, "/html/body/form[1]")          # Absolute path  
driver.find_element(By.XPATH, "//form[@id='loginForm']")      # By attribute  
driver.find_element(By.XPATH, "//input[@name='username']")    # Input with name username
```

Locating Hyperlinks by Link Text

Use this when the full text of a hyperlink is known.

Example HTML:

```
<a href="continue.html">Continue</a>  
<a href="cancel.html">Cancel</a>
```

Selenium:

```
continue_link = driver.find_element(By.LINK_TEXT, "Continue")
```

Example:

Click the "Forgot password?" link on any login page:

```
driver.find_element(By.LINK_TEXT, "Forgot password?").click()
```

Locating by Partial Link Text

Returns the first link containing the specified partial text.

Selenium:

```
link = driver.find_element(By.PARTIAL_LINK_TEXT, "Conti")
```

Locating Elements by Tag Name

Use this to locate elements based on HTML tag names such as h1, p, li, etc.

Example HTML:

```
<h1>Welcome</h1>
```

```
<p>Site content goes here.</p>
```

Selenium:

```
heading = driver.find_element(By.TAG_NAME, "h1")
```

Locating Elements by Class Name

Use this when locating an element using the class attribute.

Example HTML:

```
<p class="content">Site content goes here.</p>
```

Selenium:

```
content = driver.find_element(By.CLASS_NAME, "content")
```

Locating Elements by CSS Selector

CSS Selectors are extremely powerful and fast.

They support:

- Class selectors (.classname)

- ID selectors (#id)
- Attribute selectors ([attr=value])
- Hierarchy selectors (div p, ul > li)
- Pseudo selectors (not fully supported in Selenium)

Example HTML:

<p class="content">Site content goes here.</p>

Selenium:

```
content = driver.find_element(By.CSS_SELECTOR, "p.content")
```

Example:

Locating Amazon search box by CSS selector:

```
search = driver.find_element(By.CSS_SELECTOR, "input#twotabsearchtextbox")
```

Strategy	Example	Best Use Case
ID	By.ID	When the element has a unique ID
NAME	By.NAME	Simple forms with name attributes
XPATH	By.XPATH	Complex elements without IDs
LINK_TEXT	By.LINK_TEXT	When link text is known
PARTIAL_LINK_TEXT	By.PARTIAL_LINK_TEXT	When link text is long or dynamic
TAG_NAME	By.TAG_NAME	Lists, tables, headings
CLASS_NAME	By.CLASS_NAME	Reusable classes in HTML
CSS_SELECTOR	By.CSS_SELECTOR	Fast, flexible locators

Waits in Selenium

Modern web applications often use AJAX to load content dynamically. This means that when a page loads, some elements may not appear immediately.

If Selenium tries to locate such elements too early, it can raise exceptions like:

- NoSuchElementException
- ElementNotVisibleException

Waits help Selenium pause until the required condition is met.

Selenium provides two types of waits:

- **Explicit Wait**
- **Implicit Wait**

Explicit Waits

An explicit wait tells Selenium to wait **until a specific condition occurs** before moving forward. This is more precise and is used for elements that load slowly or appear after AJAX calls.

Example:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
driver = webdriver.Chrome()
driver.get("http://somedomain/url_that_delays_loading")
```

try:

```
    element = WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.ID, "myDynamicElement"))
    )
```

finally:

```
    driver.quit()
```

Explanation:

- Selenium waits **up to 10 seconds**
- Checks every **500 milliseconds**
- If the element appears → returns element
- If it doesn't appear → throws `TimeoutException`

Example (Explicit Wait)

Wait for the Amazon search results to load:

```
wait = WebDriverWait(driver, 10)
```

```
results = wait.until(  
    EC.visibility_of_element_located((By.CSS_SELECTOR, "div.s-main-slot"))  
)
```

This ensures Selenium waits until the results section is visible before interacting with it.

Common Expected Conditions

Selenium provides many built-in conditions:

- `title_is`
- `title_contains`
- `presence_of_element_located`
- `visibility_of_element_located`
- `visibility_of`
- `presence_of_all_elements_located`
- `text_to_be_present_in_element`
- `text_to_be_present_in_element_value`
- `frame_to_be_available_and_switch_to_it`
- `invisibility_of_element_located`
- `element_to_be_clickable`
- `staleness_of`

- `element_to_be_selected`
- `element_located_to_be_selected`
- `alert_is_present`

Example usage:

```
wait = WebDriverWait(driver, 10)
```

```
element = wait.until(EC.element_to_be_clickable((By.ID, "someid")))
```

Example (Element to Be Clickable)

Wait for the "Place Order" button to be clickable before proceeding:

```
wait = WebDriverWait(driver, 10)
```

```
place_order = wait.until(  
    EC.element_to_be_clickable((By.ID, "place-order"))  
)  
place_order.click()
```

Custom Wait Conditions

If the predefined conditions are insufficient, you can create custom wait logic by defining a class with a `__call__` method.

Example:

```
class element_has_css_class(object):  
  
    def __init__(self, locator, css_class):  
        self.locator = locator  
        self.css_class = css_class  
  
    def __call__(self, driver):  
        element = driver.find_element(*self.locator)  
        if self.css_class in element.get_attribute("class"):  
            return element
```

```
else:  
    return False
```

Usage:

```
wait = WebDriverWait(driver, 10)  
element = wait.until(  
    element_has_css_class((By.ID, 'myNewInput'), "myCSSClass")  
)
```

Real-World Example (Custom Wait Condition)

Wait until a loading spinner turns into the “active” class:

```
wait.until(  
    element_has_css_class((By.ID, "loadingSpinner"), "active")  
)
```

Useful when websites show animated loaders while content is still loading.

Implicit Waits

An implicit wait tells WebDriver to keep trying to locate an element for a set number of seconds. Once set, it applies to **all find_element calls** for the life of the driver.

Example:

```
driver = webdriver.Chrome()  
driver.implicitly_wait(10)  
driver.get("http://somedomain/url_that_delays_loading")  
myDynamicElement = driver.find_element(By.ID, "myDynamicElement")
```

Explanation:

- Selenium waits up to **10 seconds**
- Polls repeatedly until the element appears
- Applies automatically to all subsequent element searches

Example (Implicit Wait)

Loading Gmail inbox after login:

```
driver.implicitly_wait(15)
```

```
driver.find_element(By.XPATH, "//*[@text()='Inbox']")
```

Even if Gmail takes 10 seconds to load the inbox, Selenium will keep trying until the element becomes available.

